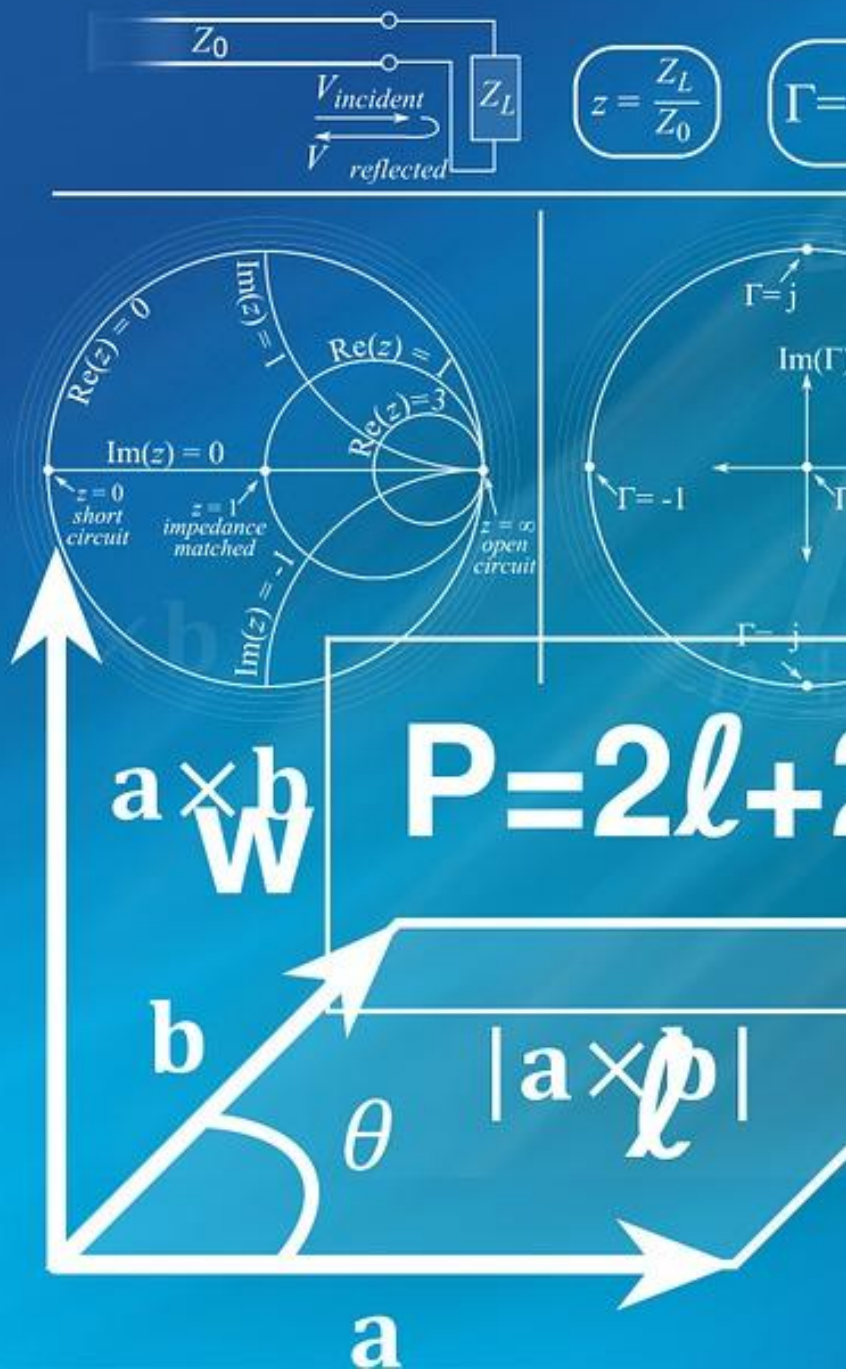




集成学习算法



目录

contents

01

个体与集成

02

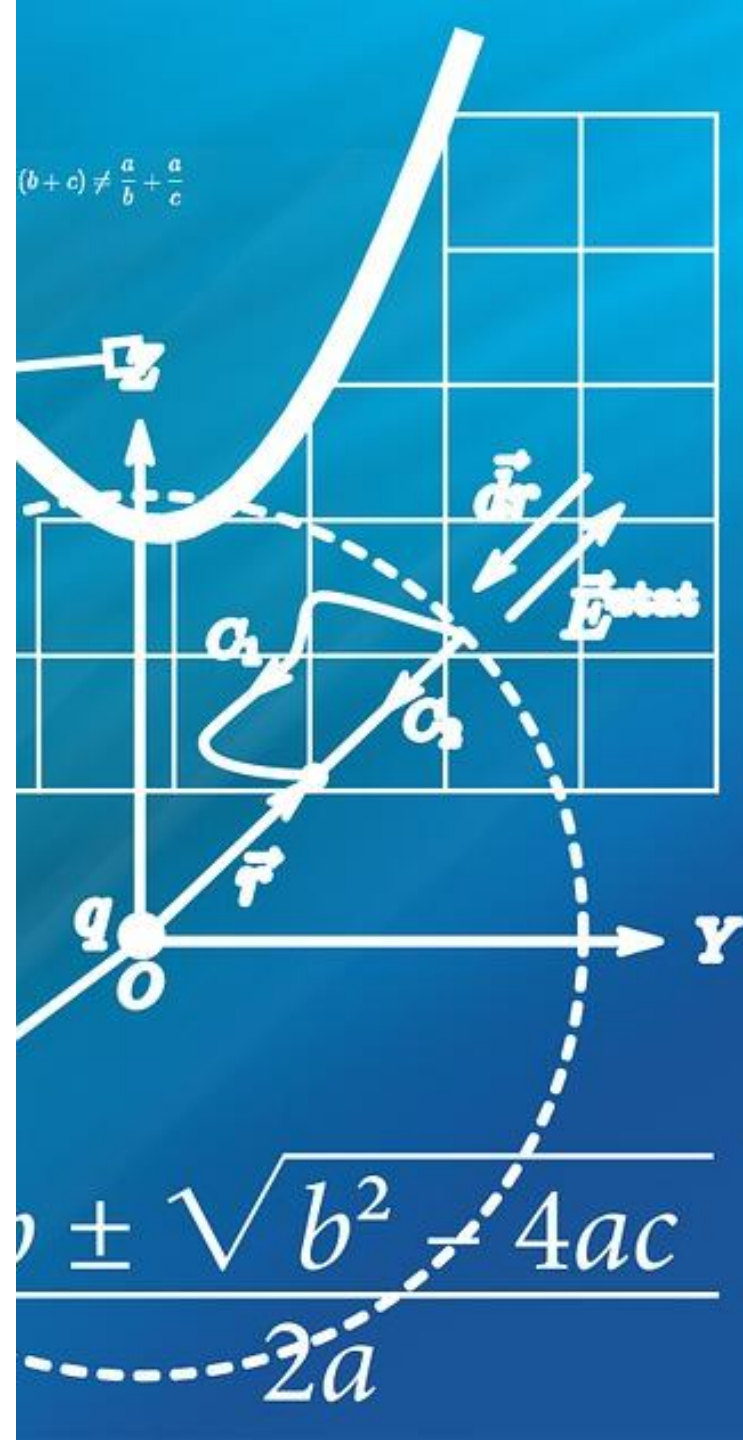
Bagging算法随机森林

03

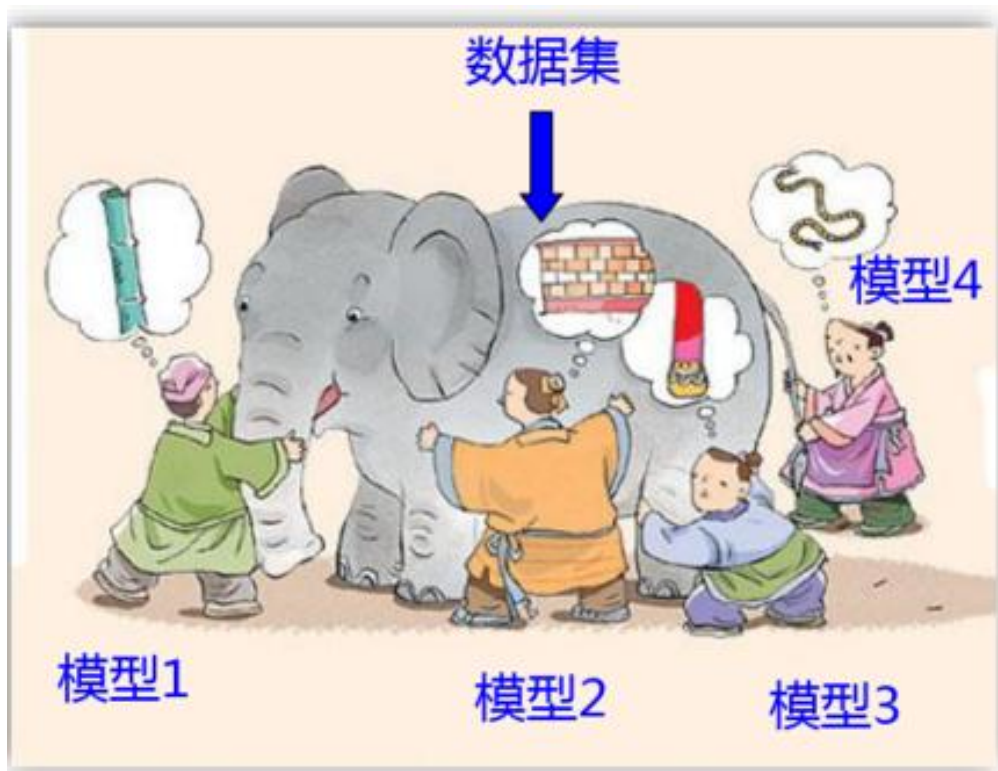
Boosting算法

01

个体与集成



6.1 个体与集成-集成学习思想



1) 个体学习模型：有如盲人摸象，从单一视角探索未知数据集，只有一个结果。

2) 集成学习模型：集思广益，从不同的视角探索未知数据集，存在多个学习过程，允许一个或者多个学习结果。

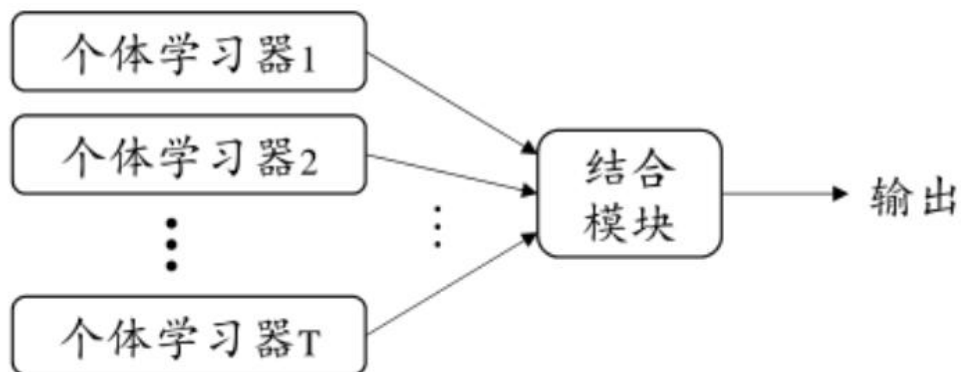
三个臭皮匠，胜过诸葛亮！



6.1 个体与集成-集成学习动机

- 1) 在机器学习中，直接建立一个高性能的分类器是很困难的。
- 2) 如果能找到一系列性能较差的分类器，并把它们集成起来的话，也许就能得到更好的分类器。
- 3) 日常生活中，所谓的民主决策，便是部分的利用了这个想法。
- 4) 譬如选总统，每个人都以自己的考虑，投下自己的一票，但最后由多数人选出的总统，似乎好于由一个人指定的总统。
- 5) 集成学习最早也叫做“Committee Voting Method”，就是因为它和投票的过程相似。

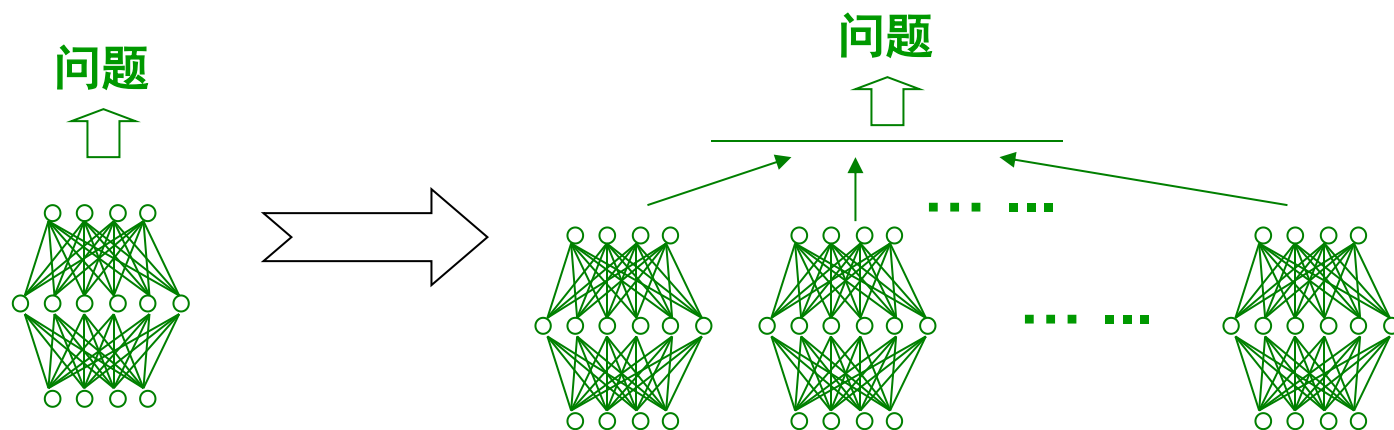
6.1 个体与集成-集成学习示意图



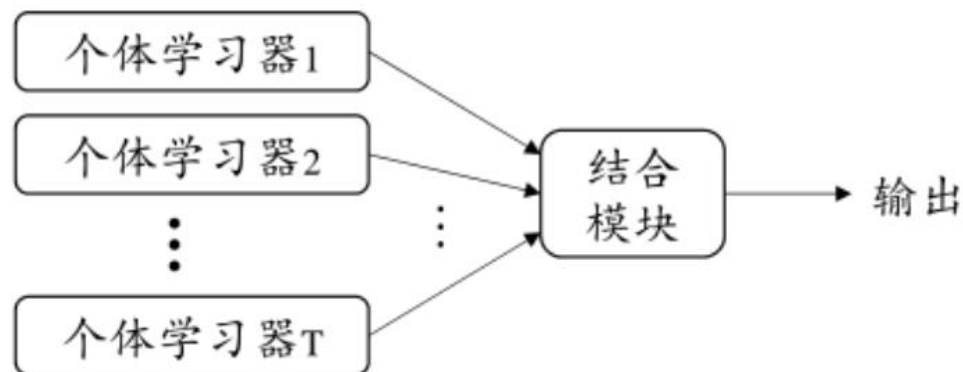
1) **集成学习的定义**：是一种机器学习范式，通过构建并结合多个学习器来完成学习任务，解决同一个问题。

2) **集成学习的结构**：先产生一组“个体学习器”，再用某种策略将它们结合起来。个体学习器通常由一个现有的学习算法从训练数据产生。

3) 集成学习中使用的多个学习器称为个体学习器，当个体学习器均为决策树是，称为“决策树集成”；当个体学习器均为神经网络时，称为“神经网络集成”…… ..



6.1 个体与集成-集成学习两种类型



从个体学习器类别来看，集成学习通常分为两种类型：

1. 「**同质**」**集成**，在一个集成学习中，「个体学习器」是同一类型，如「决策树集成」所有个体学习器都为决策树。
2. 「**异质**」**集成**，在一个集成学习中，「个体学习器」为不同类型，如一个集成学习中可以包含决策树模型也可以包含支持向量机模型。



6.1 个体与集成-个体越多越好吗？

既然多个个体的集成比单个个体更好，那么是不是个体越多越好？

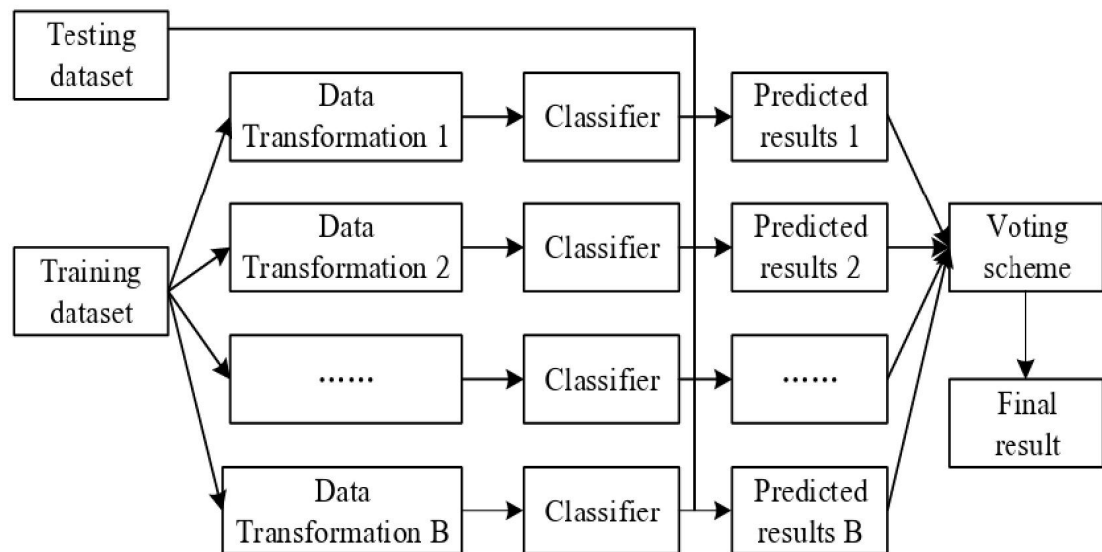
更多的个体意味着：

- 在预测时需要更大的计算开销，因为要计算更多的个体预测
- 更大的存储开销，因为有更多的个体需要保存。

个体的增加将使得个体间的差异越来越难以获得。

6.1 个体与集成-例子

➤ 分类集成学习模型



测试例1	测试例2	测试例3	测试例1	测试例2	测试例3	测试例1	测试例2	测试例3			
h_1	√	√	×	h_1	√	√	×	h_1	√	×	×
h_2	×	√	√	h_2	√	√	×	h_2	×	√	×
h_3	√	×	√	h_3	√	√	×	h_3	×	×	√
集群	√	√	√	集群	√	√	×	集群	×	×	×
(a) 集群提升性能			(b) 集群不起作用			(c) 集群起负作用					

例子： 在二分类问题中，假定3个分类器在3个样本中的表现如上图所示，其中 ✓ 表示分类正确，✗表示分类错误，集成的结果通过投票产生。

结论： 要获得好的集成，个体学习器应“好而不同”，即个体学习器要有一定的“准确性”，即学习器不能太坏，并且要有“多样性”。



6.1 个体与集成-集成学习的构造

- 1) 弱学习机 (Weaker Learner) :准确率不高, 对于一定分布的训练样本给出假设 (仅仅强于随机猜测)
- 2) 强学习机 (Strong Learner) : 准确率很高的学习算法。
- 3) 是否可以将弱学习算法提升为强学习算法?

如何产生并结合“好而不同”的个体学习器, 恰好是集成学习研究的核心

目前集成学习方法大致可分为两大类: 即个体学习器间存在强依赖关系, 必须串行生成的序列化方法, 以及个体学习器间不存在强依赖关系, 可同时生成的并行化方法; 前者的代表是**Boosting**, 后者的代表是**Bagging**和“随机森林” (Random Forest) 。

6.1 个体与集成-集成学习的目标

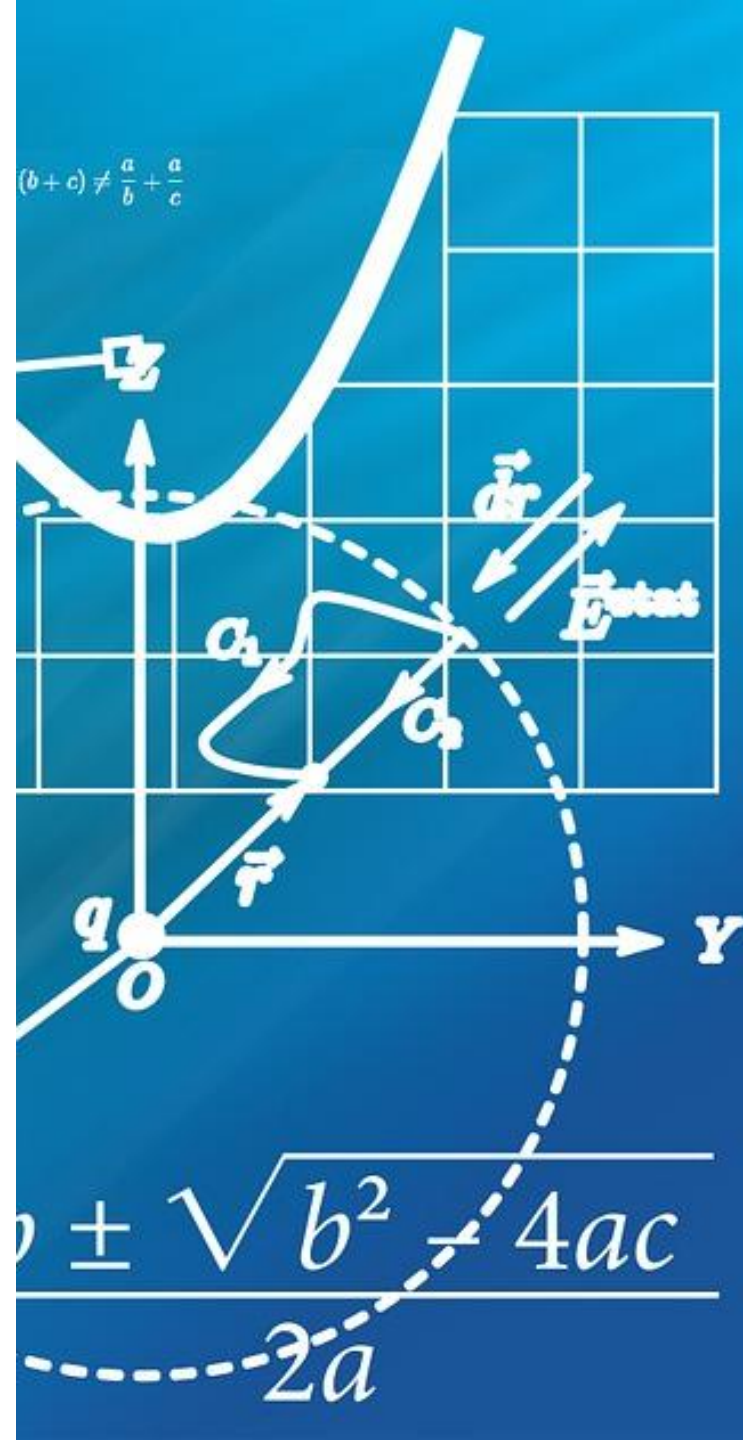
集成学习算法会考虑多个评估器的建模结果，汇总后得到一个综合的结果，以此来获取比单个模型更好的回归或分类表现。

多个模型集成成为的模型叫做集成评估器 (ensemble estimator)，组成集成评估器的每个模型都叫做基评估器 (base estimator)。通常来说，有三类集成算法：装袋法 (Bagging)，提升法 (Boosting) 和 stacking。



02

Bagging 与随机森林



6.2 Bagging与随机森林-数据集自助采样

改变数据集来构造不同的学习器，然后再把它们集成起来；

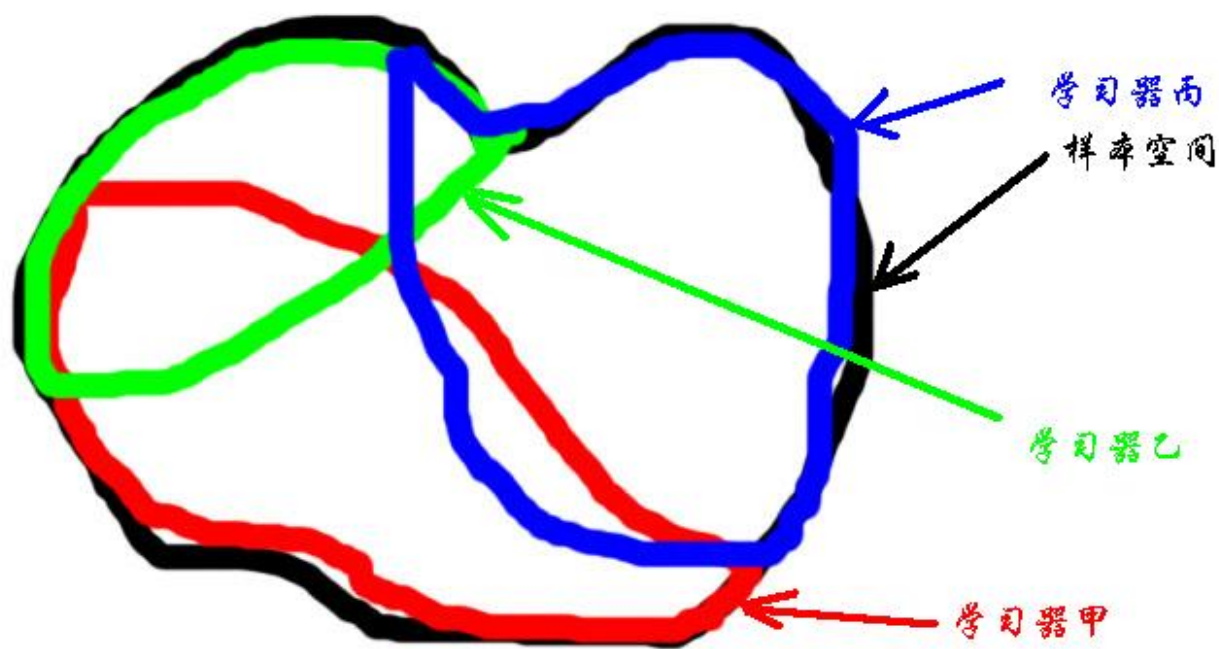
在原来的训练集上随机采样，可以得到新的训练集。

A sample of a single classifier on an imaginary set of data.	
(Original) Training Set	
Training-set-1:	1, 2, 3, 4, 5, 6, 7, 8

A sample of Bagging on the same data.	
(Resampled) Training Set	
Training-set-1:	2, 7, 8, 3, 7, 6, 3, 1
Training-set-2:	7, 8, 5, 6, 4, 2, 7, 1
Training-set-3:	3, 6, 2, 7, 5, 6, 2, 2
Training-set-4:	4, 5, 1, 4, 6, 4, 3, 8

6.2 Bagging与随机森林-Bagging算法

基本思想：对训练集有放回地抽取训练样例，从而为每一个基本分类器都构造出一个跟训练集相当大小但各不相同的训练集，从而训练出不同的基本分类器；该算法是基于对训练集进行处理的集成方法中最简单、最直观的一种。对其预测进行平均或者多数表决原则来决定评估结果。袋装法的代表模型是：随机森林。



6.2 Bagging与随机森林-Bagging算法

Bagging算法步骤:

- 从大小为 n 的原始数据集 D 中独立随机地抽取 n' 个数据($n' \leq n$), 形成一个自助数据集;
- 重复上述过程, 产生出多个独立的自助数据集;
- 利用每个自助数据集训练出一个“分量分类器”;
- 最终的分类结果由这些“分量分类器”各自的判别结果投票决定;
- 如果分类预测时, 出现了两个类收到同样的票数的情形, 最简单的做法是随机选择一个。

输入: 训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
基学习算法 $\mathcal{L}$;
训练轮数 T .

过程:

1: **for** $t = 1, 2, \dots, T$ **do**
2: $h_t = \mathcal{L}(D, \mathcal{D}_{bs})$
3: **end for**

输出: $H(\mathbf{x}) = \arg \max_{y \in \mathcal{Y}} \sum_{t=1}^T \mathbb{I}(h_t(\mathbf{x}) = y)$

6.2 Bagging与随机森林-随机森林

随机森林是Bagging的一个扩展变体。随机森林在以决策树为基学习器构建Bagging集成的基础上，进一步在决策树的训练过程中引入了随机属性选择。

区别：

传统决策树：在选择划分属性时是在当前结点的属性集合（假定有 d 个属性）中选择一个最优属性；

随机森林：对基决策树的每个结点，先从该结点的属性集合随机选择一个包含 k 个属性的子集，然后再从这个子集中选择一个最优属性用于划分。这里的参数 k 控制了随机性的引入程度，一般情况下，推荐值 $k=\log_2 d$ 。

随机森林算法

Input: Data set $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
Feature subset size K .

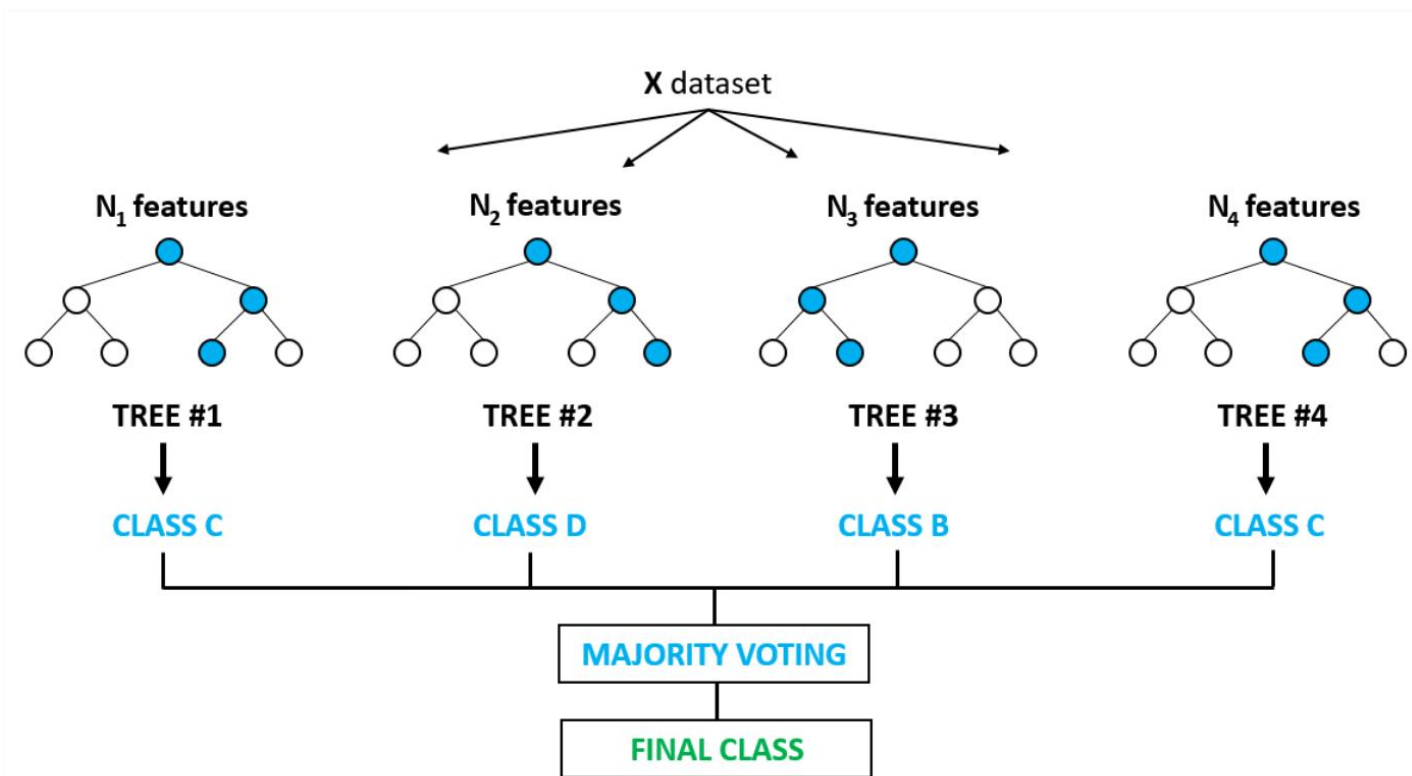
Process:

1. $N \leftarrow$ create a tree node based on D ;
2. **if** all instances in the same class **then return** N
3. $\mathcal{F} \leftarrow$ the set of features that can be split further;
4. **if** $\mathcal{F}$ is empty **then return** N
5. $\tilde{\mathcal{F}} \leftarrow$ select K features from $\mathcal{F}$ randomly;
6. $N.f \leftarrow$ the feature which has the best split point in $\tilde{\mathcal{F}}$;
7. $N.p \leftarrow$ the best split point on $N.f$;
8. $D_l \leftarrow$ subset of D with values on $N.f$ smaller than $N.p$;
9. $D_r \leftarrow$ subset of D with values on $N.f$ no smaller than $N.p$;
10. $N_l \leftarrow$ call the process with parameters (D_l, K) ;
11. $N_r \leftarrow$ call the process with parameters (D_r, K) ;
12. **return** N

Output: A random decision tree

6.2 Bagging与随机森林-随机森林示意图

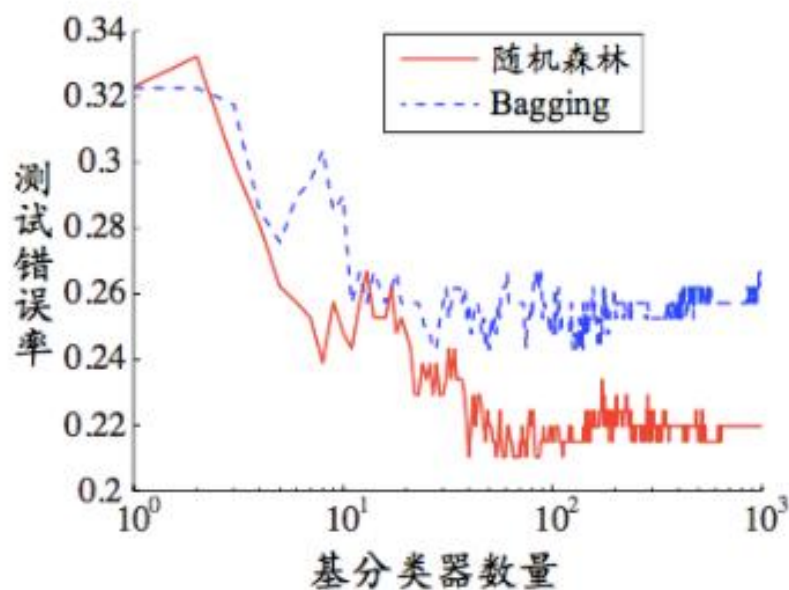
随机森林的思想就是将一个大的数据集使用自助采样法进行处理，即从原样本数据集中随机抽取多个子样本集，并基于每一个子样本集生成相应的决策树。这样，就可以构建出由许多小决策树组形成的决策树「森林」。最后，实验通过投票法选择决策树最多的预测结果作为最终的输出。



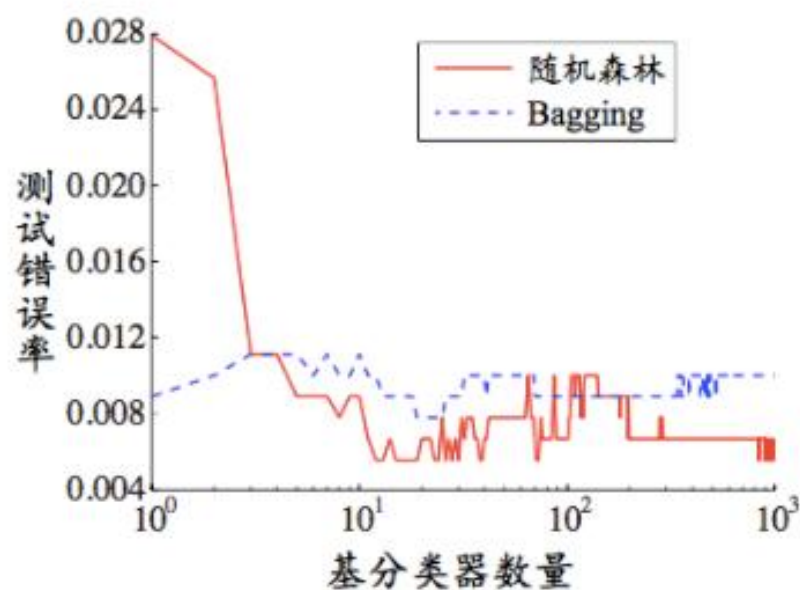
随机森林的名称来源就是「随机抽样 + 决策树森林」！

6.2 Bagging与随机森林-随机森林的收敛性

随机森林的收敛性与Bagging相似，如下图所示，随机森林的起始性能往往相对较差，特别是在集成中只包含一个基学习器时。这很容易理解，因为通过引入属性扰动，随机森林中个体学习器的性能往往有所降低。然而，随着个体学习器数目的增加，随机森林通常会收敛到更低的泛化误差。值得一提的是，随机森林的效率常优于Bagging算法。



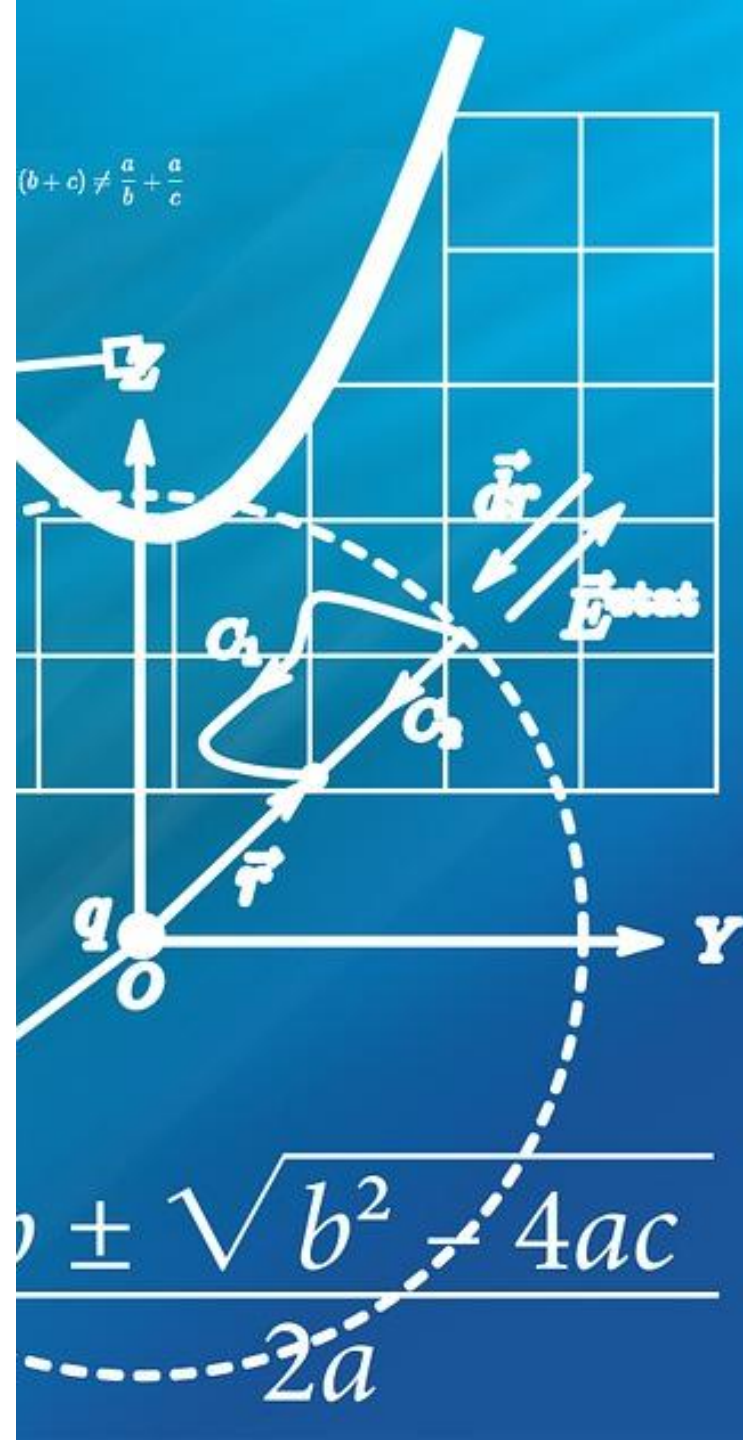
(a) glass 数据集



(b) auto-mpg 数据集

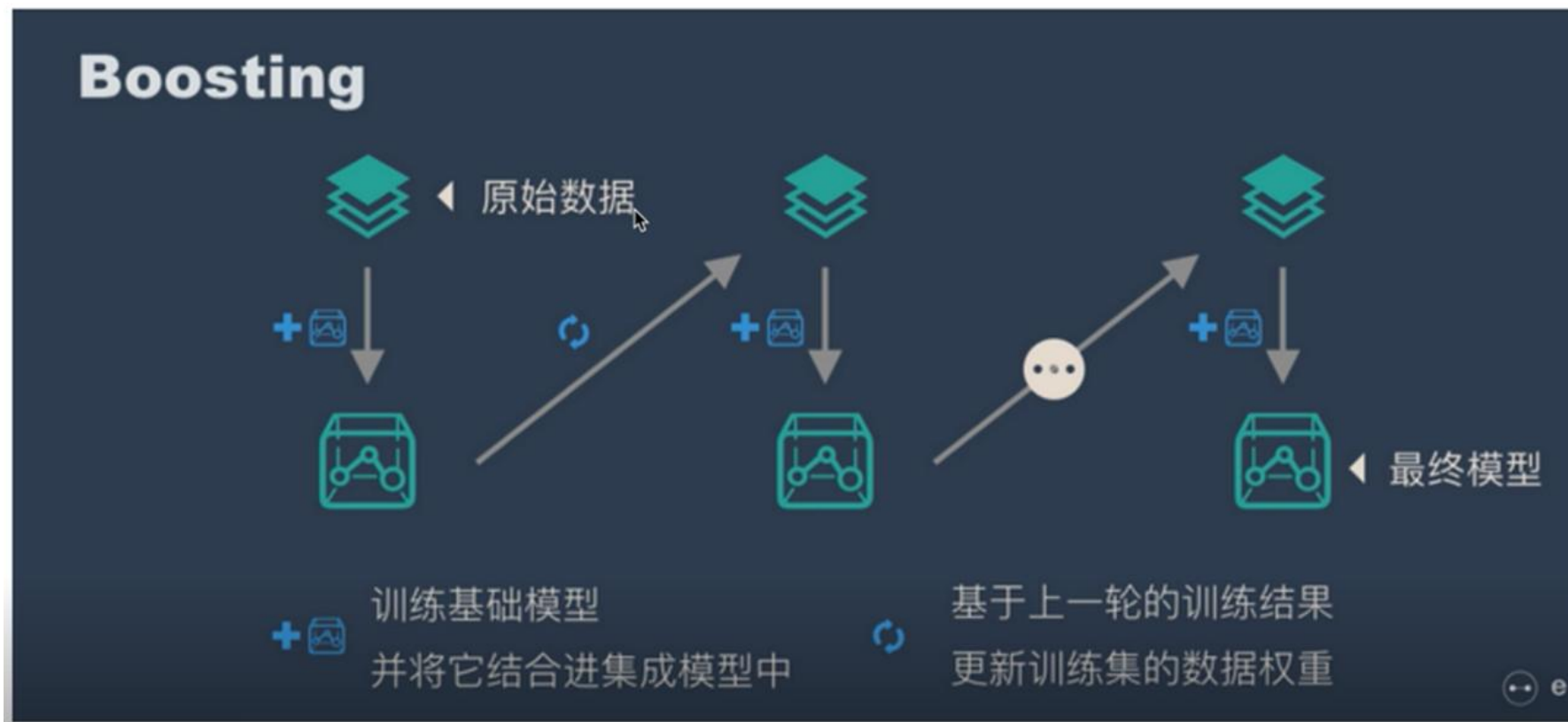


Boosting算法



1.3 Boosting算法-核心思想

Boosting算法核心思想：是结合弱评估器的力量一次次对难以评估的样本进行预测，从而构成一个强评估器。提升法的代表模型有Adaboost，它是一种改进的Boosting算法，是Boosting算法中的优秀代表。



1.3 Boosting算法-算法步骤

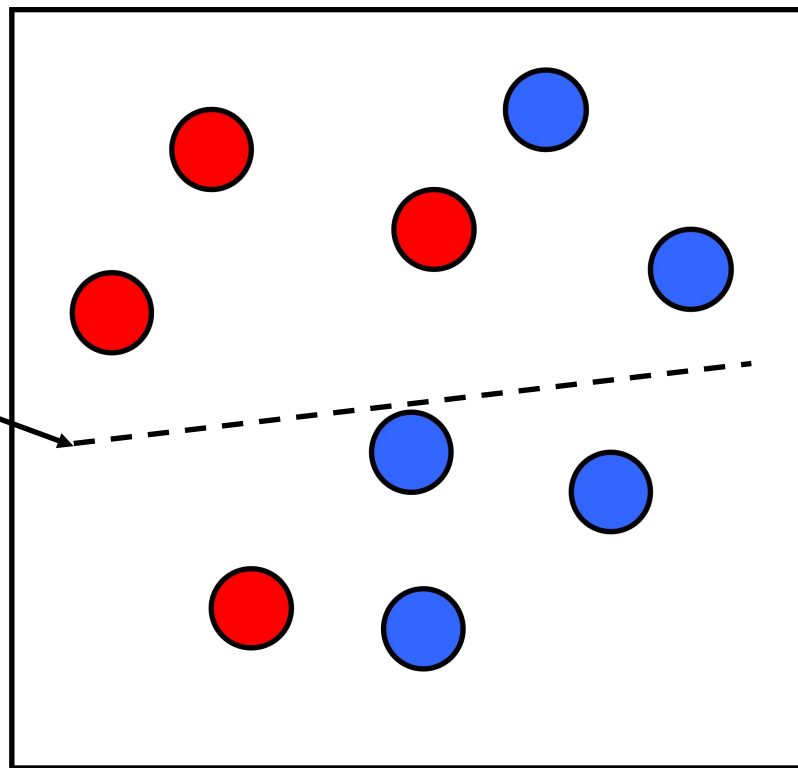
Boosting算法：将一系列相互依赖的个体学习器提升为强学习器的算法，其思想如下：首先，从训练集用初始权重训练出个体学习器1，根据个体学习器1的误差率表现来更新训练样本的权重，加大在个体学习器1中误差率大的训练样本的权重，使得这些**误差大的样本**在个体学习器2中能**得到更多的关注**；接着，基于调整权重后的训练集来训练个体学习器2；如此重复，直到个体学习器的数量达到事先指定的数目T，最后，根据某种结合策略整合这些个体学习器，得到最终的学习器。

- Step1: 原始训练集输入，带有原始分布
- Step2: 给出训练集中各样本的权重
- Step3: 将改变分布后的训练集输入已知的弱学习机，弱学习机对每个样本给出假设
- Step4: 对此次的弱学习机给出权重
- Step5: 转到Step2, 直到循环到达一定次数或者某度量标准符合要求
- Step6: 将弱学习机按其相应的权重加权组合形成强学习机



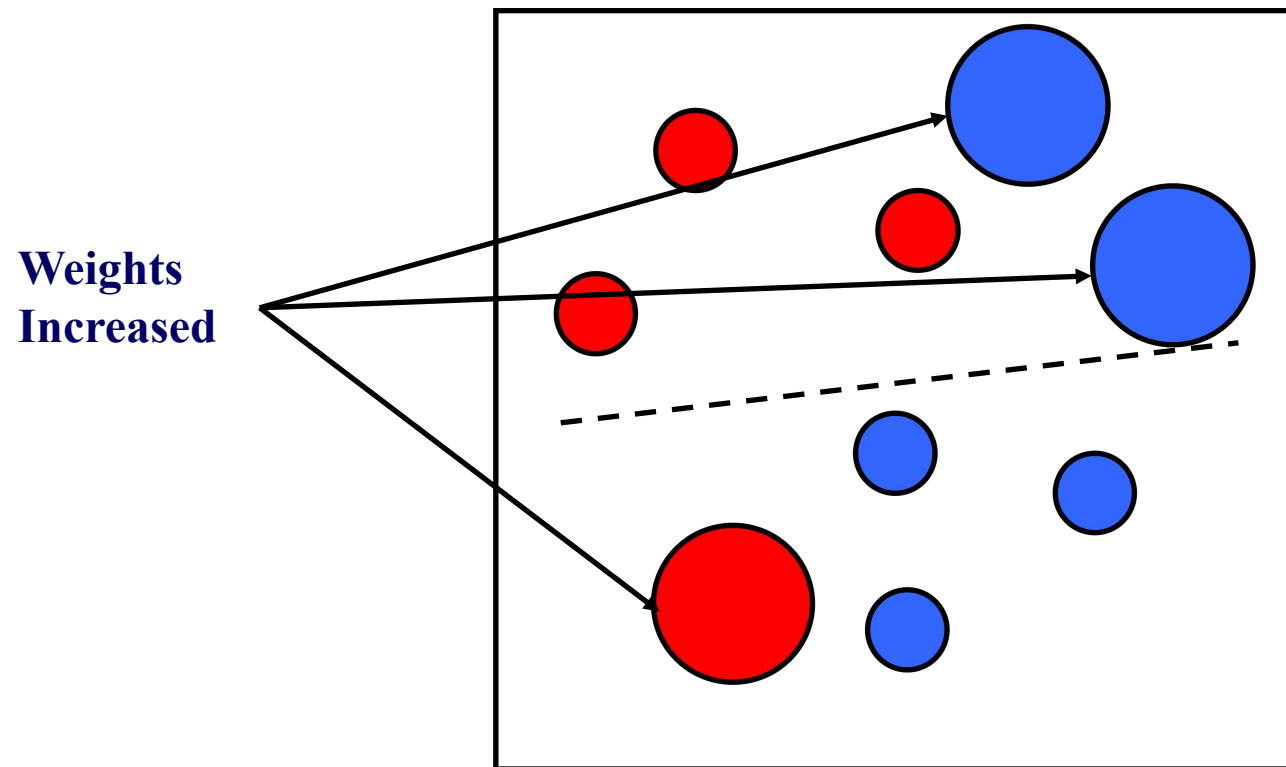
1.3 Boosting算法-图示

Weak
Classifier 1



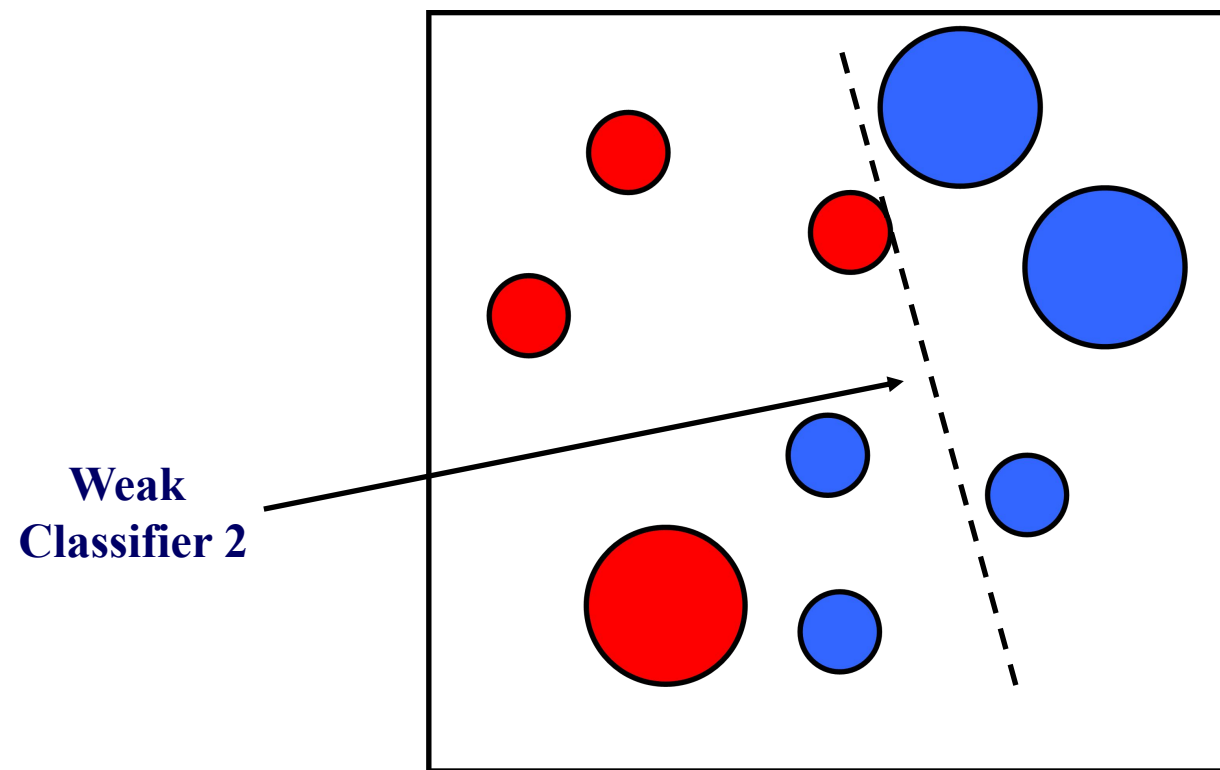


1.3 Boosting算法-图示



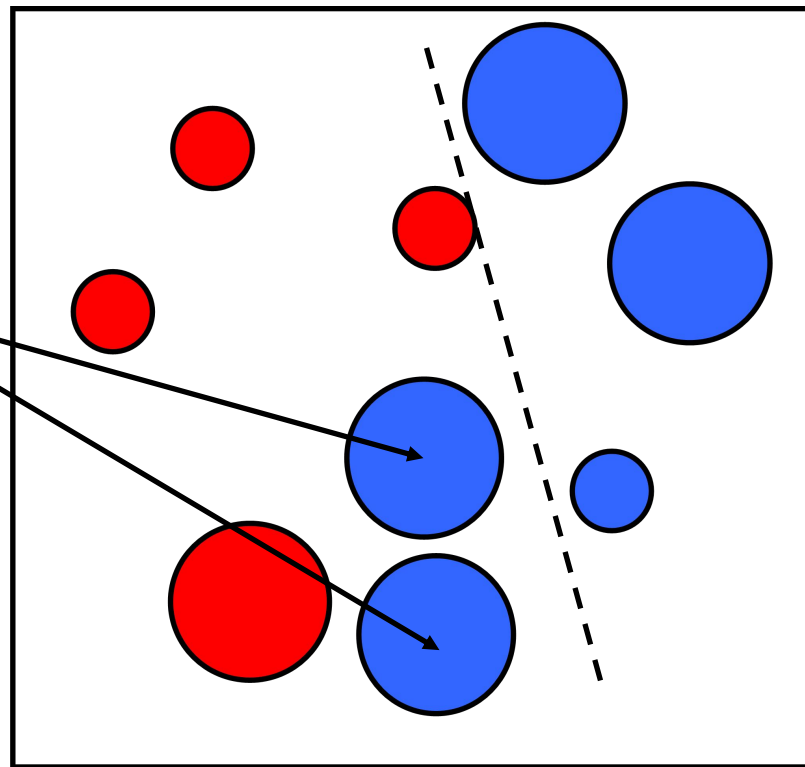


1.3 Boosting算法-图示



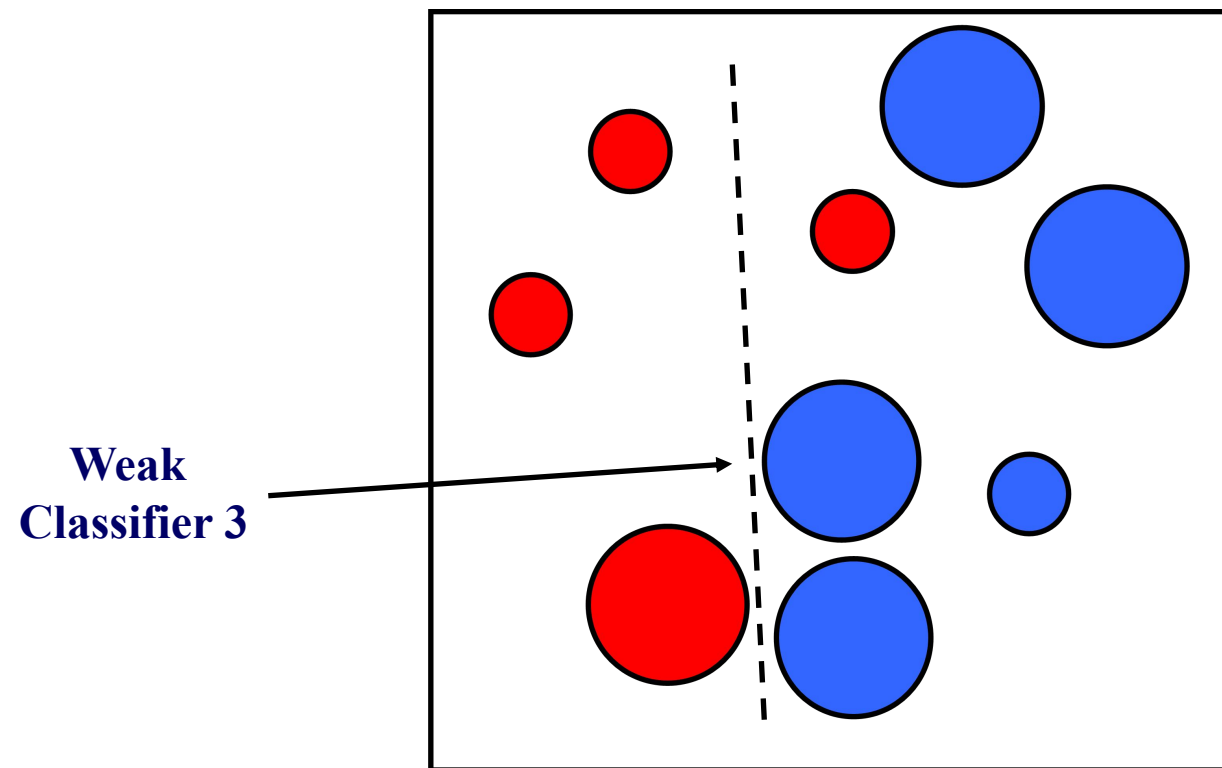
1.3 Boosting算法-图示

Weights
Increased



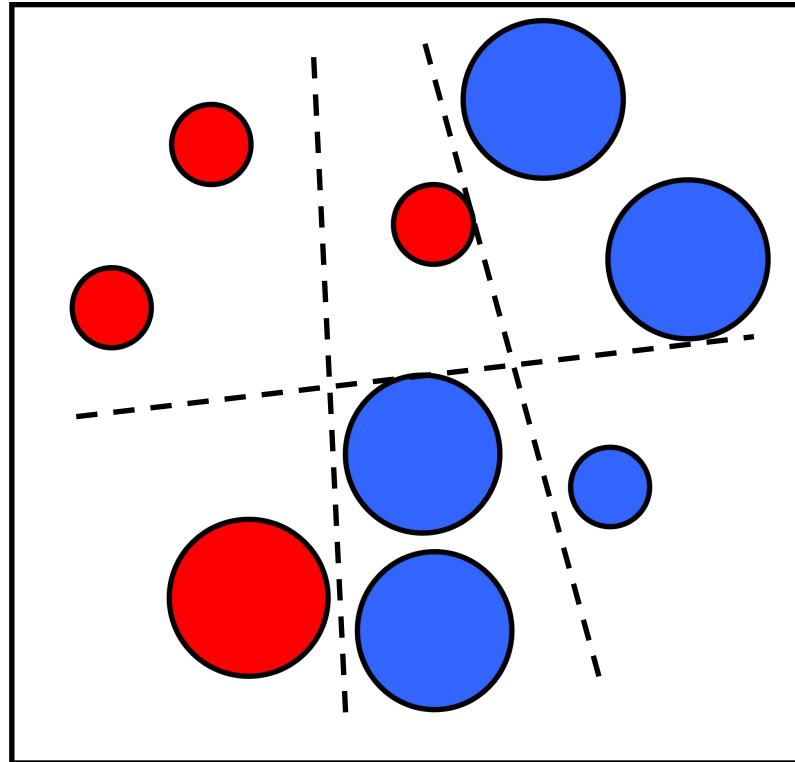


1.3 Boosting算法-图示



1.3 Boosting算法-图示

**Final classifier is
a combination of weak
classifiers**





1.3 Boosting算法-核心思想

样本的权重

- 没有先验知识的情况下，初始的分布应为等概率分布，也就是训练集如果有 N 个样本，每个样本的分布概率为 $1/N$
- 每次循环一后提高错误样本的分布概率，分错样本在训练集中所占权重增大，使得下一次循环的弱学习机能够集中力量对这些错误样本进行判断。

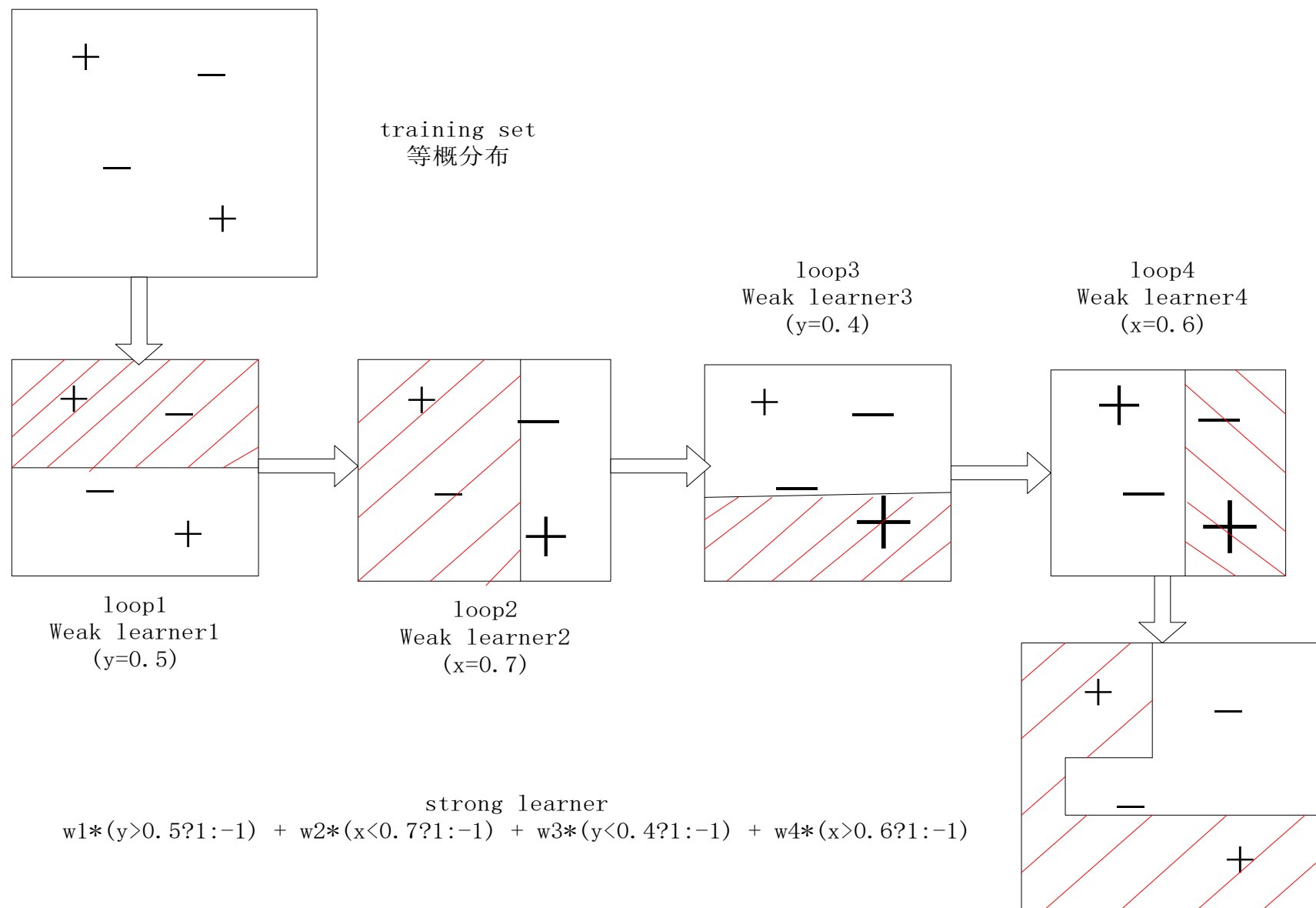
弱学习机的权重

- 准确率越高的弱学习机权重越高

循环控制：损失函数达到最小

- 在强学习机的组合中增加一个加权的弱学习机，使准确率提高，损失函数值减小。

1.3 Boosting算法-问题演示 (Boosting训练过程)





1.3 Boosting算法-Bagging和Boosting的区别

训练集:

- Bagging:随机选择,各轮训练集相互独立
- Boosting:各轮训练集并不独立,它的选择与前轮的学习结果有关

预测函数:

- Bagging:没有权重;可以并行生成
- Boosting:有权重;只能顺序生成

1.3 Boosting算法-Adaboost算法流程图

AdaBoosting算法是一种改进的Boosting算法。该算法除了提高前面个体学习器中错分样本的权重，使其在后面的个体学习器中被更加“关注”，还使用加权投票法集成个体学习器，对准确率高的个体学习器加大权重，使其在表决中起较大的作用；对准确率低的个体学习器降低权重，使其在表决中起较小的作用。

输入：训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
基学习算法 $\mathfrak{L}$;
训练轮数 T .

过程:

- 1: $\mathcal{D}_1(\mathbf{x}) = 1/m$.
- 2: **for** $t = 1, 2, \dots, T$ **do**
- 3: $h_t = \mathfrak{L}(D, \mathcal{D}_t)$;
- 4: $\epsilon_t = P_{\mathbf{x} \sim \mathcal{D}_t}(h_t(\mathbf{x}) \neq f(\mathbf{x}))$;
- 5: **if** $\epsilon_t > 0.5$ **then break**
- 6: $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$;
- 7:
$$\mathcal{D}_{t+1}(\mathbf{x}) = \frac{\mathcal{D}_t(\mathbf{x})}{Z_t} \times \begin{cases} \exp(-\alpha_t), & \text{if } h_t(\mathbf{x}) = f(\mathbf{x}) \\ \exp(\alpha_t), & \text{if } h_t(\mathbf{x}) \neq f(\mathbf{x}) \end{cases}$$
$$= \frac{\mathcal{D}_t(\mathbf{x}) \exp(-\alpha_t f(\mathbf{x}) h_t(\mathbf{x}))}{Z_t}$$
- 8: **end for**

输出: $H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(\mathbf{x}) \right)$

1.3 Boosting算法-Adaboost算法流程图

AdaBoosting算法流程：

（一）初始化训练数据的权重

每一个训练样本最开始时都被赋予相同的权重： $1/N$ ， 即

$$D_1 = (\omega_{11}, \omega_{12}, \omega_{13}, \dots, \omega_{1N})$$

其中， $\omega_{1i} = \frac{1}{N}, i = 1, 2, \dots, N$

D_1 表示第一个弱分类器的权重分布，

ω_{1i} 表示每一个样本的权重大小， i 是当前训练数据集的样本个数。

1.3 Boosting算法-Adaboost算法流程图

AdaBoosting算法流程：

(二) 计算学习误差率

当模型进行多次迭代后，我们用 $k = 1, 2, \dots, K$ 表示进行了 K 次迭代

1. 使用具有权重分布 D_K 的训练数据集进行学习，得到基本分类器 $G_k(x) : X \rightarrow \{-1, +1\}$

$\{-1, +1\}$ 表示分类型标签的集合， X 表示 x 样本实例所属实例空间。

2. 计算 $G_k(x)$ 在训练集上的分类误差率

$$e_K = P(G_K(x_i) \neq y_i) = \sum_{i=1}^N \omega_{ki} I(G_k(x_i) \neq y_i)$$

这里应用了概率论中数学期望的概念， $E = \sum x_i p_i$

由上述式子可知： $G_k(x)$ 在训练数据集上的误差率 e_K 就是被模型 $G_k(x)$ 误分类样本的权值之和



1.3 Boosting算法-Adaboost算法流程图

AdaBoosting算法流程：

(三) 计算模型 $G_k(x)$ 的权重系数

$$\alpha_k = \frac{1}{2} \ln \frac{1 - e_k}{e_k}$$

由上述式子可知，当 $e_k \leq \frac{1}{2}$ ， $\alpha_k \geq 0$ ，且 α_k 随着 e_k 的减少而增大，意味着分类误差率越小的基分类器在最终分类器的作用越大。

1.3 Boosting算法-Adaboost算法流程图

AdaBoosting算法流程：

(四) 更新样本权重D

$$D_{k+1} = (\omega_{k+1,1}, \omega_{k+1,2}, \dots, \omega_{k+1,N})$$

$$\omega_{k+1,i} = \frac{\omega_{k,i}}{Z_k} \exp(-\alpha_k y_i G_k(x_i)), \text{ 其中 } i = 1, 2, \dots, N$$

这一步骤的目的是为了得到样本的权值分布，并用于下一次迭代。我们将基分类器 $G_k(x)$ 的误分类样本的权重增大，而被正确分类的样本权重减少。通过这种形式，让Adaboost算法能聚焦于较难分的样本上。

其中 Z_k 称为规范化因子，使得模型 D_{k+1} 成为一个概率分布：

$$Z_k = \sum_{i=1}^N \omega_{k,i} \exp(-\alpha_k y_i G_k(x_i))$$

从 $\omega_{k+1,i}$ 计算公式可以看出：如果第 i 个样本分类错误，则 $y_i G_k(x_i) < 0$ ，继而会导致样本的权重在第 $k+1$ 个弱分类器中增大；如果分类正确，则权重在第 $k+1$ 个弱分类器中降低。

1.3 Boosting算法-Adaboost算法流程图

AdaBoosting算法流程：

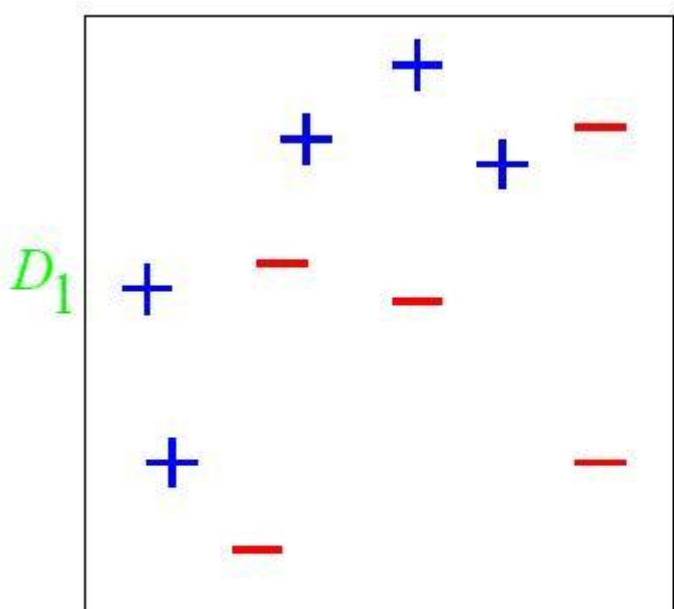
(五) 组合各个弱分类器

$$f(x) = \sum_{k=1}^K \alpha_k G_k(x)$$

从而得到最终分类器，如下：

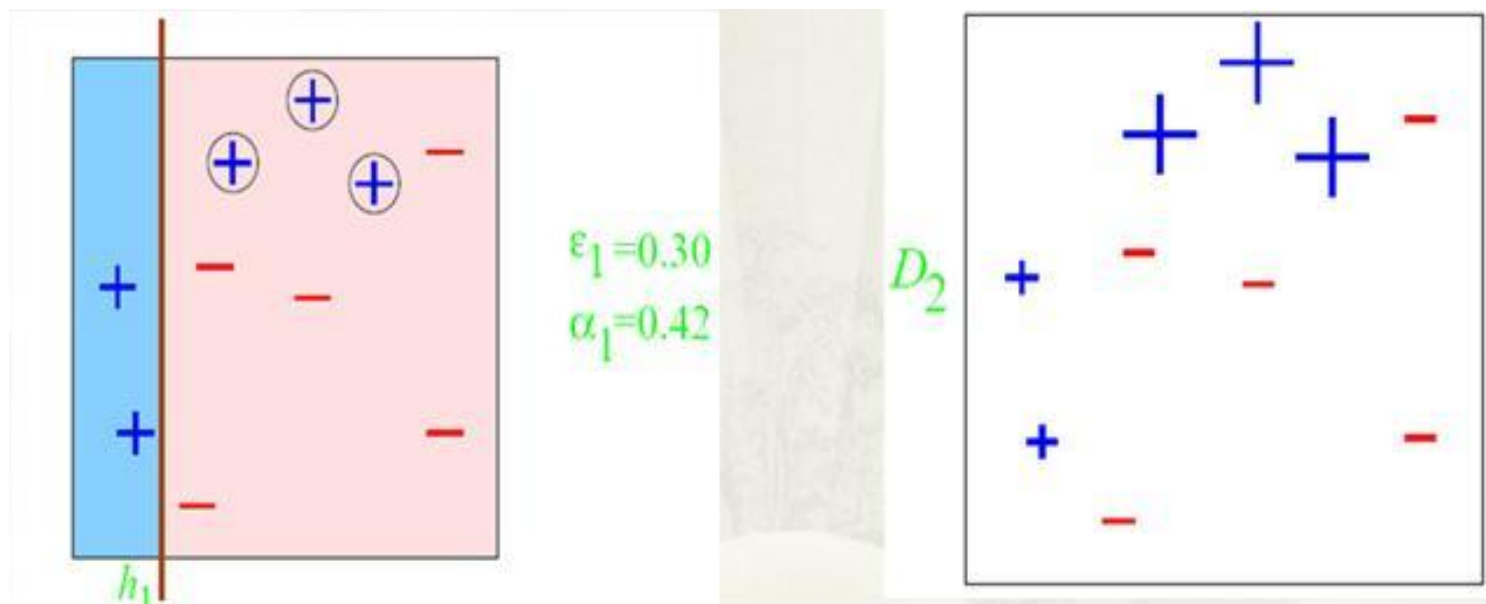
$$G(x) = \text{sign}(f(x)) = \text{sign}\left(\sum_{k=1}^K \alpha_k G_k(x)\right)$$

1.3 Boosting算法-Adaboost算法实现过程



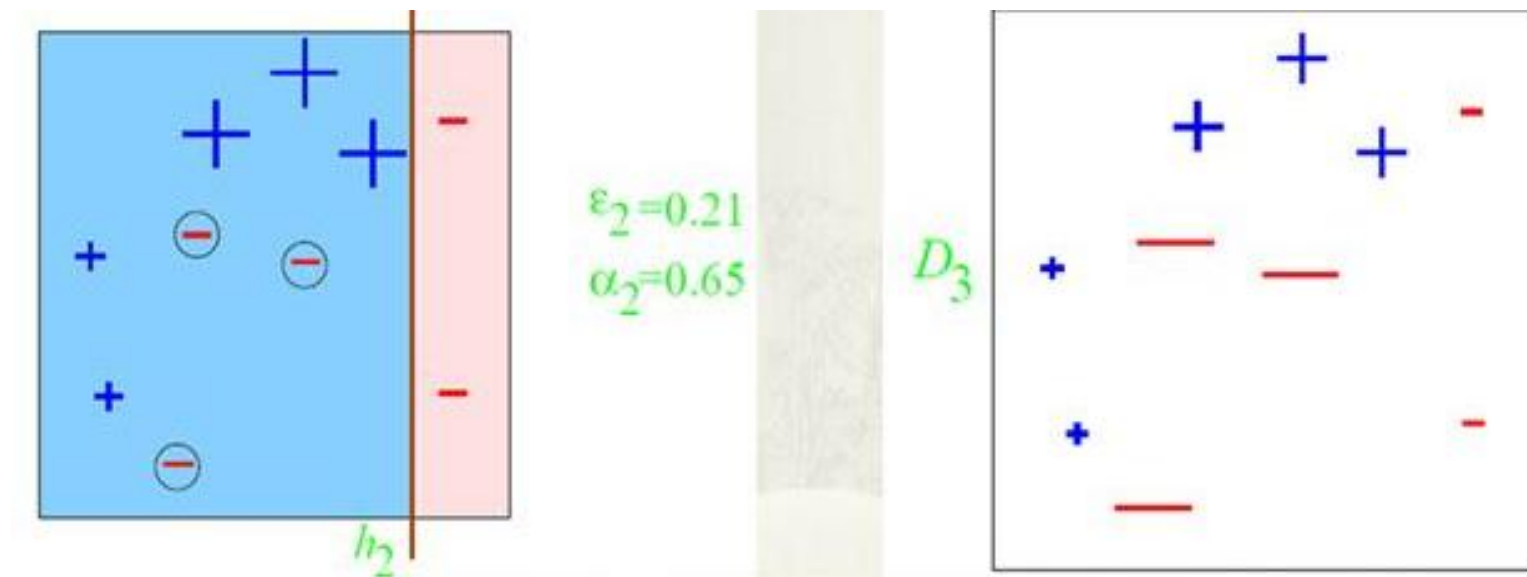
图中，“+”和“-”分别表示两种类别，在这个过程中，我们使用水平或者垂直的直线作为分类器，来进行分类。

1.3 Boosting算法-Adaboost算法实现过程



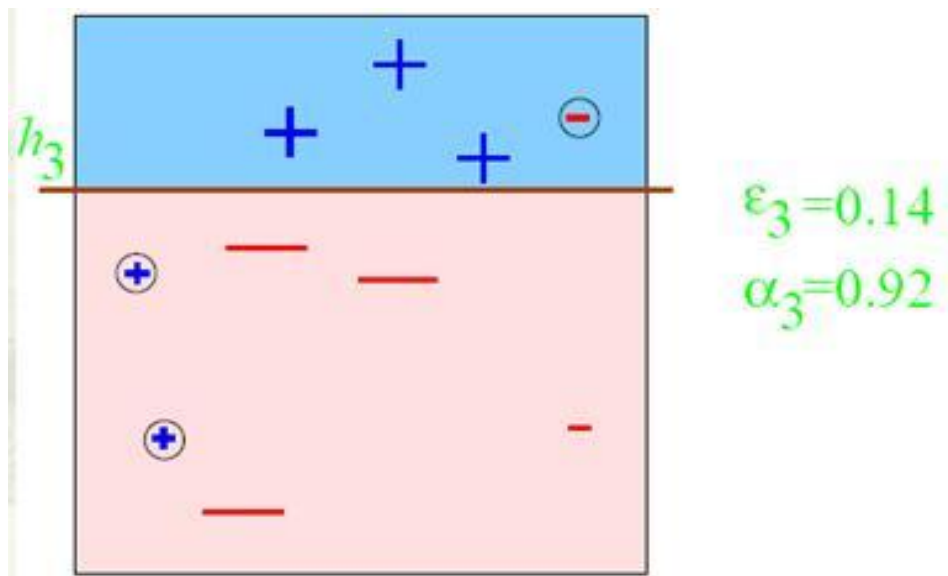
第一步：根据分类的正确率，得到一个新的样本分布 D_2 ，一个子分类器 h_1 ，其中划圈的样本表示被分错的。在右边的图中，比较大的“+”表示对该样本做了加权。

1.3 Boosting算法-Adaboost算法实现过程



第二步：根据分类的正确率，得到一个新的样本分布**D3**，一个子分类器**h2**。

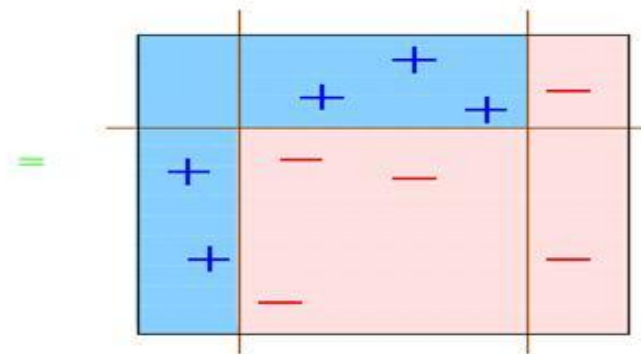
1.3 Boosting算法-Adaboost算法实现过程



第三步：得到一个子分类器 h_3

1.3 Boosting算法-Adaboost算法实现过程

整合所有子分类器：

$$H_{\text{final}} = \text{sign} \left(0.42 \begin{array}{|c|} \hline \text{blue} \\ \hline \text{red} \\ \hline \end{array} + 0.65 \begin{array}{|c|} \hline \text{blue} \\ \hline \text{red} \\ \hline \end{array} + 0.92 \begin{array}{|c|} \hline \text{blue} \\ \hline \text{red} \\ \hline \end{array} \right)$$


因此可以得到整合的结果，从结果中看，即使简单的分类器，组合起来也能获得很好的分类效果。

1.3 Boosting算法-Adaboost例子

给定如下表所示的训练数据，假设弱分类器由 $x < v$ 或 $x > v$ 产生，其阈值 v 使该分类器在训练数据集上分类误差最低。试用AdaBoost算法产生一个强分类器。

表 训练数据表

序号	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1

解 初始化数据权值分布

$$D_1 = (w_{11}, w_{12}, \dots, w_{110})$$
$$w_{1i} = 0.1, \quad i = 1, 2, \dots, 10$$

对 $m = 1$,

(a) 在权值分布为 D_1 的训练数据上，阈值 v 取 2.5 时分类误差率最低，故基本分类器为

$$G_1(x) = \begin{cases} 1, & x < 2.5 \\ -1, & x > 2.5 \end{cases}$$

1.3 Boosting算法-Adaboost例子

(b) $G_1(x)$ 在训练数据集上的误差率 $e_1 = P(G_1(x_i) \neq y_i) = 0.3$.

(c) 计算 $G_1(x)$ 的系数: $\alpha_1 = \frac{1}{2} \log \frac{1-e_1}{e_1} = 0.4236$.

(d) 更新训练数据的权值分布:

$$D_2 = (w_{21}, \dots, w_{2i}, \dots, w_{210})$$

$$w_{2i} = \frac{w_{1i}}{Z_1} \exp(-\alpha_1 y_i G_1(x_i)), \quad i=1, 2, \dots, 10$$

$$D_2 = (0.0715, 0.0715, 0.0715, 0.0715, 0.0715, 0.0715, \\ 0.1666, 0.1666, 0.1666, 0.0715)$$

$$f_1(x) = 0.4236 G_1(x)$$

$$w_{2,1} = \frac{w_1}{Z_1} \exp(-0.4236 * 1 * 1) = \frac{0.1}{0.1 * \exp(-0.4236 * 1 * 1) * 7 + 0.1 * \exp(0.4236 * 1 * 1) * 3} \\ * \exp(-0.4236) = 0.0714$$

分类器 $\text{sign}[f_1(x)]$ 在训练数据集上有 3 个误分类点.

1.3 Boosting算法-Adaboost例子

对 $m = 2$,

(a) 在权值分布为 D_2 的训练数据上, 阈值 v 是 8.5 时分类误差率最低, 基本分类器为

$$G_2(x) = \begin{cases} 1, & x < 8.5 \\ -1, & x > 8.5 \end{cases}$$

(b) $G_2(x)$ 在训练数据集上的误差率 $e_2 = 0.2143$.

(c) 计算 $\alpha_2 = 0.6496$.

(d) 更新训练数据权值分布:

$$\begin{aligned} D_3 = & (0.0455, 0.0455, 0.0455, 0.1667, 0.1667, 0.1667, \\ & 0.1060, 0.1060, 0.1060, 0.0455) \\ f_2(x) = & 0.4236G_1(x) + 0.6496G_2(x) \end{aligned}$$

分类器 $\text{sign}[f_2(x)]$ 在训练数据集上有 3 个误分类点.

1.3 Boosting算法-Adaboost例子

对 $m=3$,

(a) 在权值分布为 D_3 的训练数据上, 阈值 v 是 5.5 时分类误差率最低, 基本分类器为

$$G_3(x) = \begin{cases} 1, & x > 5.5 \\ -1, & x < 5.5 \end{cases}$$

(b) $G_3(x)$ 在训练样本集上的误差率 $e_3 = 0.1820$.

(c) 计算 $\alpha_3 = 0.7514$.

(d) 更新训练数据的权值分布:

$$D_4 = (0.125, 0.125, 0.125, 0.102, 0.102, 0.102, 0.065, 0.065, 0.065, 0.125)$$

于是得到:

$$f_3(x) = 0.4236G_1(x) + 0.6496G_2(x) + 0.7514G_3(x)$$

分类器 $\text{sign}[f_3(x)]$ 在训练数据集上误分类点个数为 0.

于是最终分类器为

$$G(x) = \text{sign}[f_3(x)] = \text{sign}[0.4236G_1(x) + 0.6496G_2(x) + 0.7514G_3(x)]$$

1.3 Boosting算法-思考题

为什么每次迭代都要把分错的点的权值变大呢？这样有什么好处呢？不这样不行吗？

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right), \quad \alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right).$$

这里面的 α 表示的权值，是关于误差的表达式，到这里就可以得到比较清晰的答案了，所有的一切都指向了误差。提高错误点的权值，当下一次分类器再次分错了这些点之后，会提高整体的错误率，这样就导致 α 变的很小，最终导致这个分类器在整个混合分类器的权值变低。也就是说，**这个算法让优秀的分类器占整体的权值更高，而差的分类器权值更低。**